MOBILE APPLICATION DEVELOPMENT

ANDROID FUNDAMENTALS MODULE

Android Location Services & Smartphone Sensors

Location APIs, Google Maps, Geofencing and the Android Sensor Framework



Learning Outcomes

01 Explain how Android applications obtain and use device location.

02 Understand Fused Location Provider, Google Maps and geofencing.

03 Explain the Android Sensor Framework and its main components.

04 Distinguish accelerometer, gyroscope and proximity sensor use cases.

05 Apply location and sensor concepts to a simple mobile application.

Lecture Roadmap

01 Location-based services and Android location APIs

02 Location permissions and current location

03 Google Maps and geofencing

04 Android Sensor Framework

05 Accelerometer, gyroscope, proximity and camera

06 Design considerations and integrated campus example

Location-Based Services

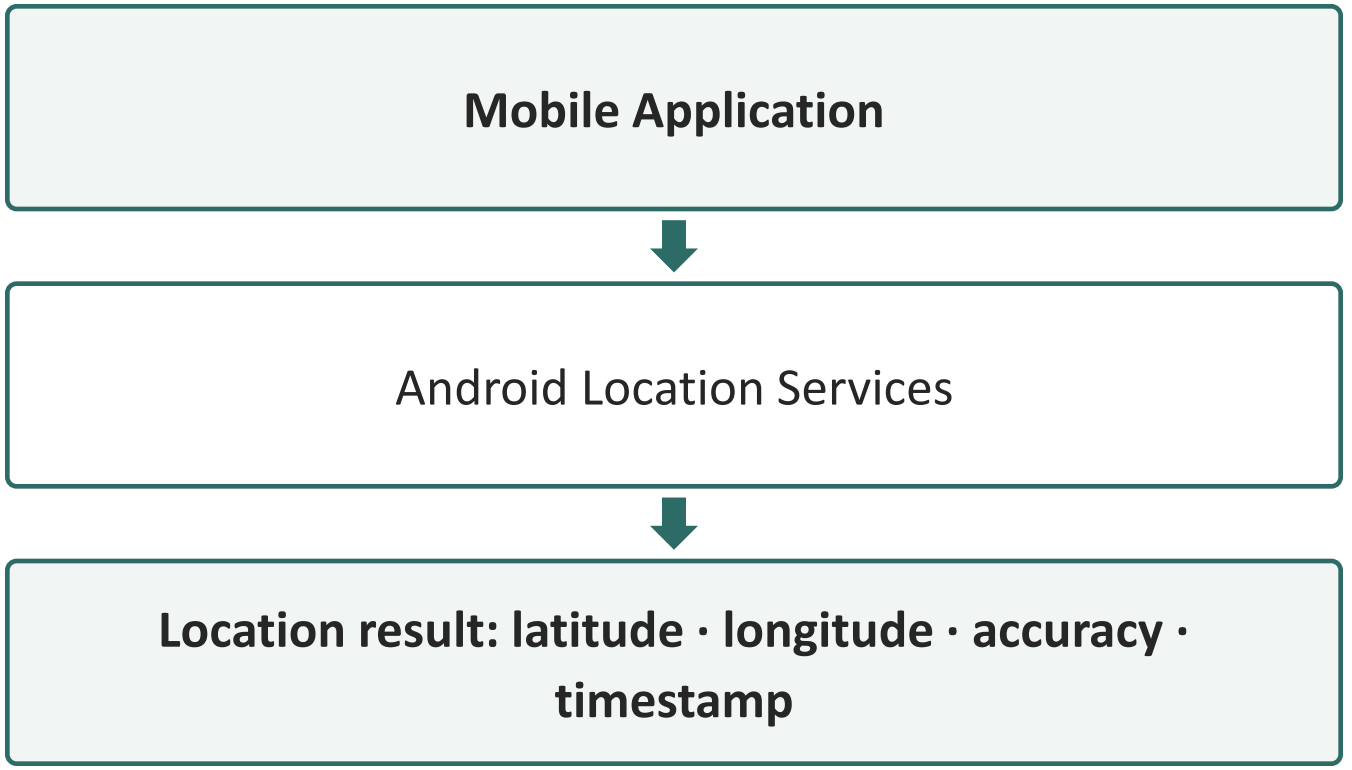
01

- Location-Based Services (LBS) use the device's geographic position to provide context-aware features.
- Common examples: navigation, ride-hailing, food delivery, fitness, weather and campus services.
- Core idea: the application receives location data and uses it to make a decision or display information.
- Example: a campus app can show the user's position and identify whether the user is inside campus.

How Does a Phone Know Its Location?

- GPS/GNSS can estimate position using signals from navigation satellites.
- Android can also use other available location sources, including network-based information.
- Applications normally do not implement satellite calculations themselves.
- Android location APIs provide a common interface for requesting and receiving location data.

Android Location Architecture



Requests location through Android location APIs

Uses available location sources

Key idea: the app consumes a location result rather than directly controlling GPS hardware.

Location APIs: What Developers Use

LocationManager

Android framework API for accessing location services.

Fused Location Provider

Google Play services API that combines available location signals.

- For many modern applications, Fused Location Provider is a convenient choice.
- Selection depends on application requirements, platform dependencies and required control.

Fused Location Provider: Basic Workflow

1. Check location permission.
2. Create/access the fused location client.
3. Request current location or location updates.
4. Receive a Location result.
5. Use latitude, longitude and accuracy in the application.

Always handle unavailable location and permission-denied cases.

Location Data

Latitude

north/south position expressed in degrees.

Longitude

east/west position expressed in degrees.

Accuracy

estimated uncertainty of the reported position.

Timestamp

when the location was obtained.

These values support maps, distance calculations, geofencing and other location-aware logic.

Location Permissions

Android protects location because it can reveal sensitive information about user movements.

ACCESS_COARSE_LOCATION

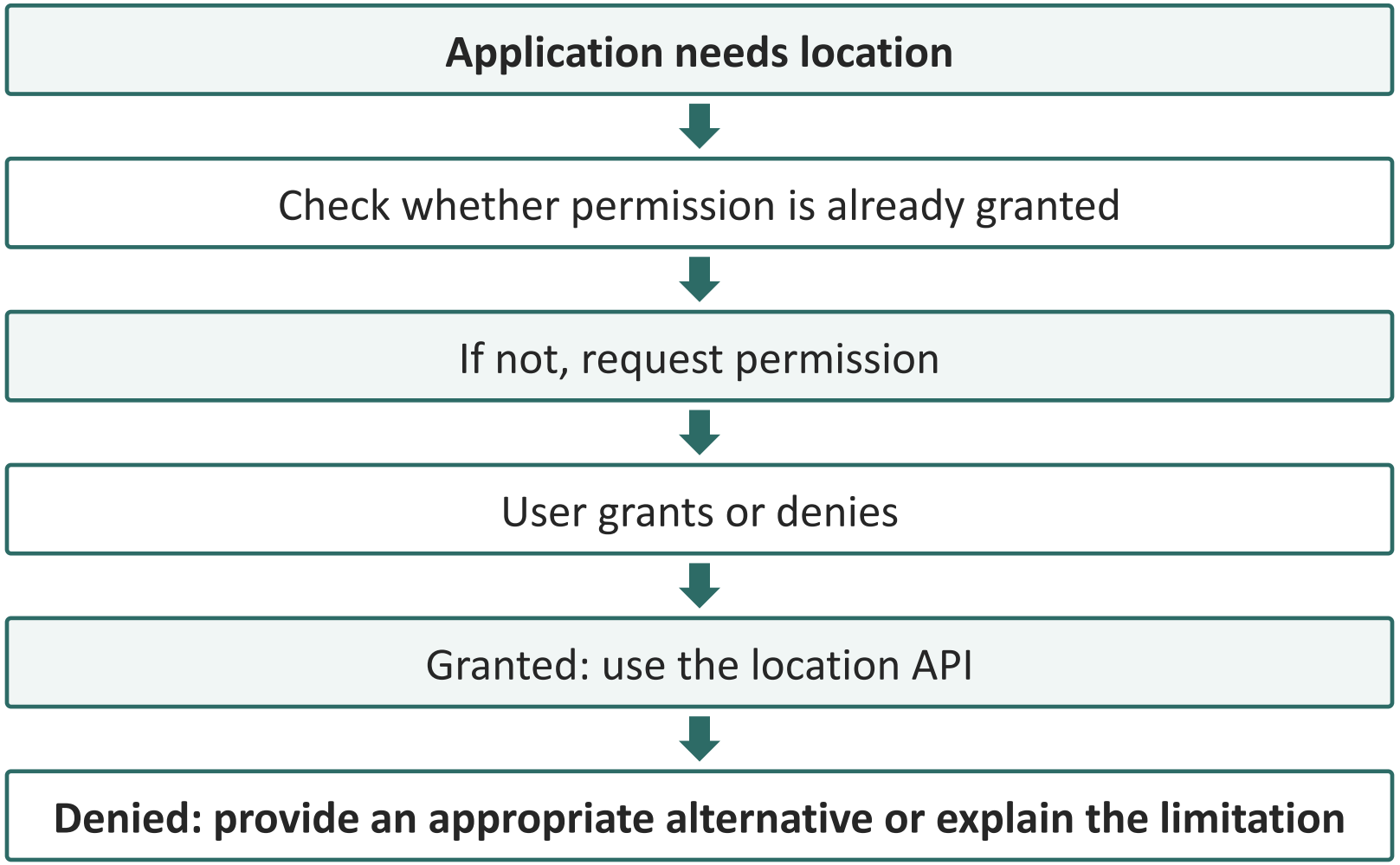
approximate location.

ACCESS_FINE_LOCATION

precise location when available and permitted.

Runtime permission must be requested when required by the app. Request only the level of access needed for the feature.

Permission Workflow



Kotlin: Requesting Location Permission

Runtime permission must be checked before any protected location call.

```
if (ActivityCompat.checkSelfPermission(this,
    Manifest.permission.ACCESS_FINE_LOCATION) !=
    PackageManager.PERMISSION_GRANTED) {
ActivityCompat.requestPermissions(this,
    arrayOf(Manifest.permission.ACCESS_FINE_LOCATION),
    LOCATION_REQUEST)
return
}
```

Important: check permission before protected location operations.

Getting the Current Location

FusedLocationProviderClient provides access to the fused location service.

1. obtain the client
2. check permission
3. request a current location
4. read latitude / longitude
5. update the UI or application state

Keep implementation focused on the API flow rather than project boilerplate.

Google Maps in Android

- Google Maps SDK for Android allows an app to display and interact with a map.
- Essential concepts: map, camera position, zoom, markers and user location.
- Location APIs provide the location; Maps provides the visual map experience.
- Use Maps when geographic information needs to be presented visually.

Location API vs. Google Maps

LOCATION API

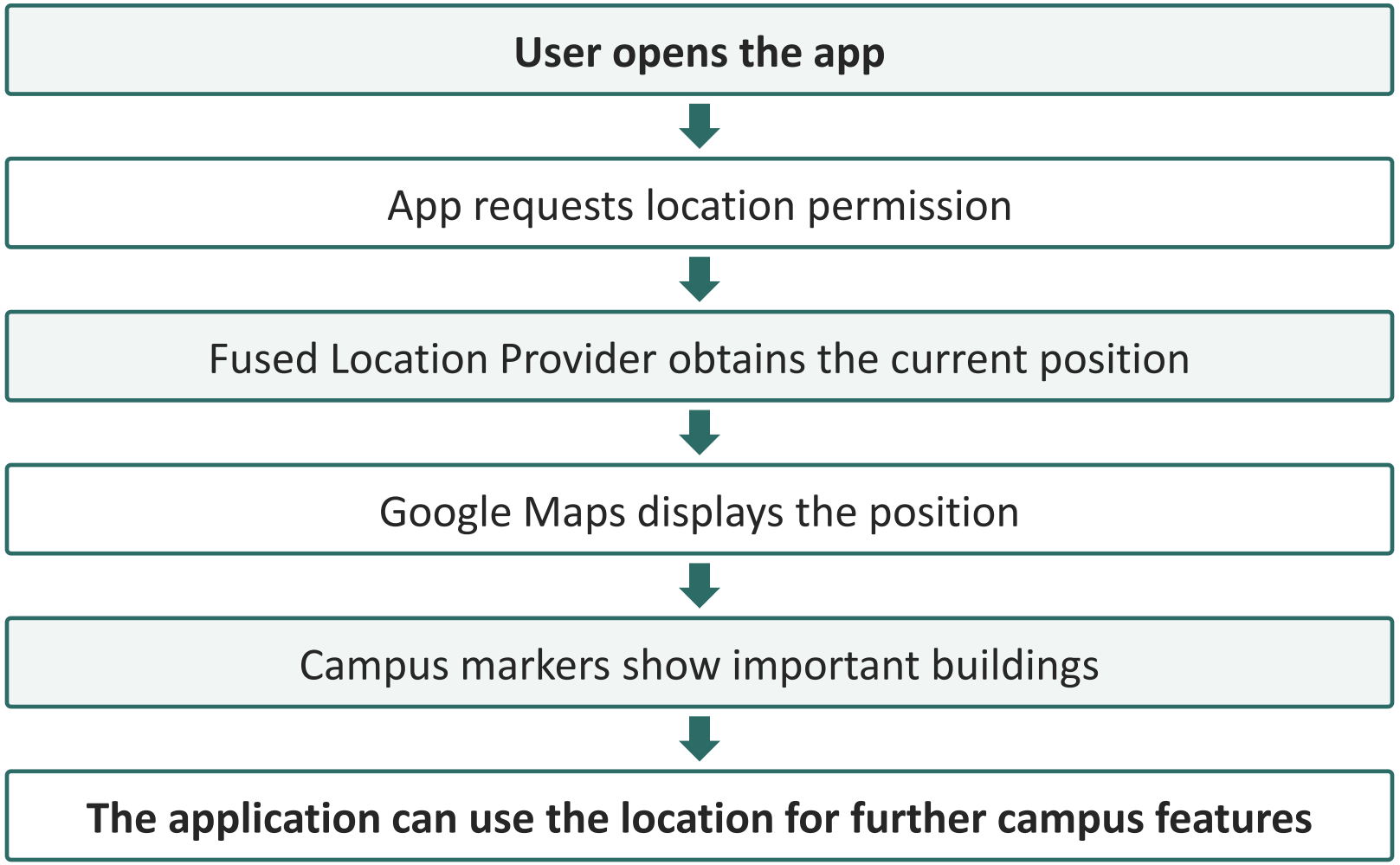
provides latitude, longitude, accuracy and other location information. Used for decisions, tracking, distance and location-aware logic.

GOOGLE MAPS SDK

displays a map and supports markers, camera movement and visual interaction.

An application can use either independently or combine both.

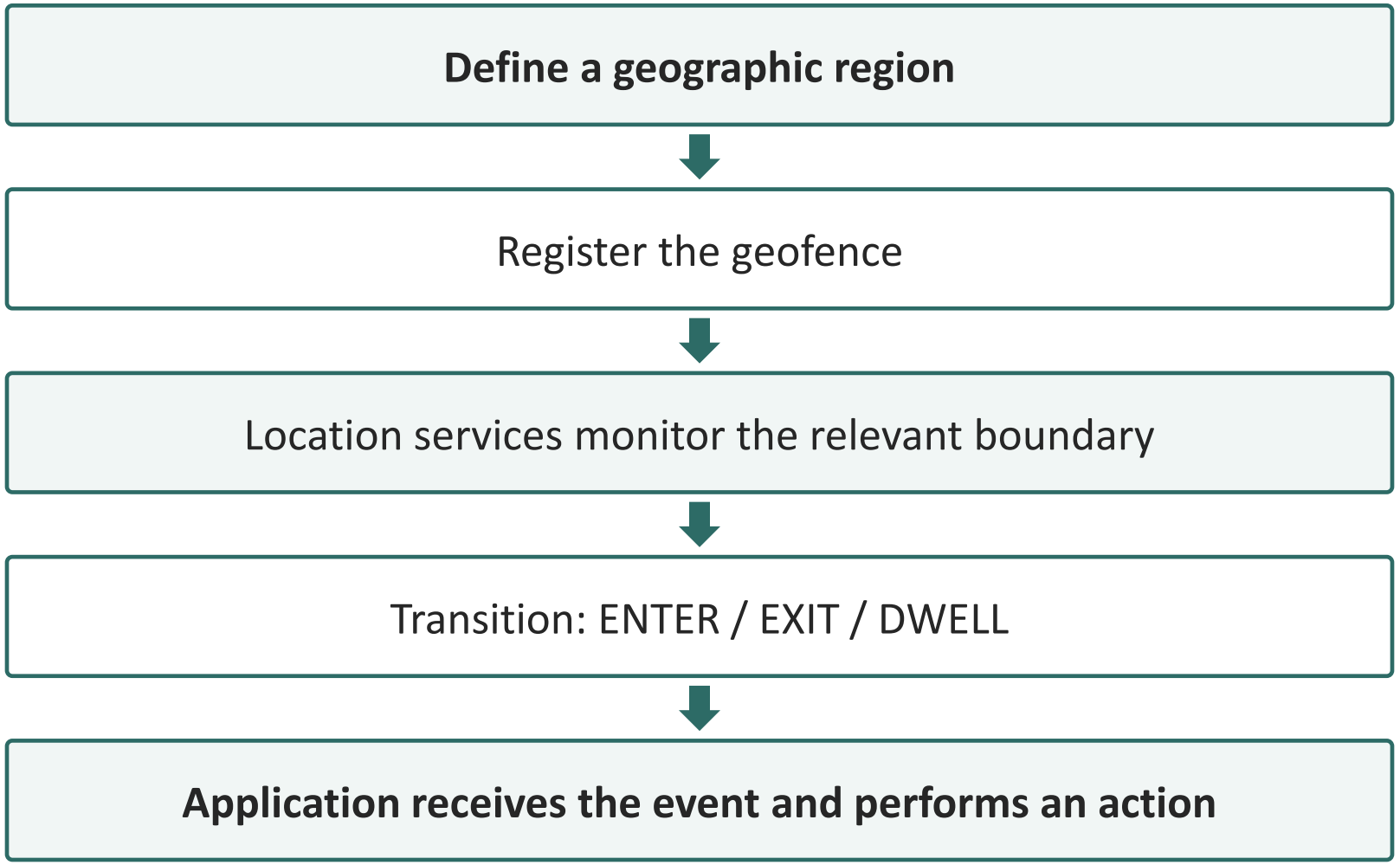
Example: University Campus Map



Geofencing: Location as an Event

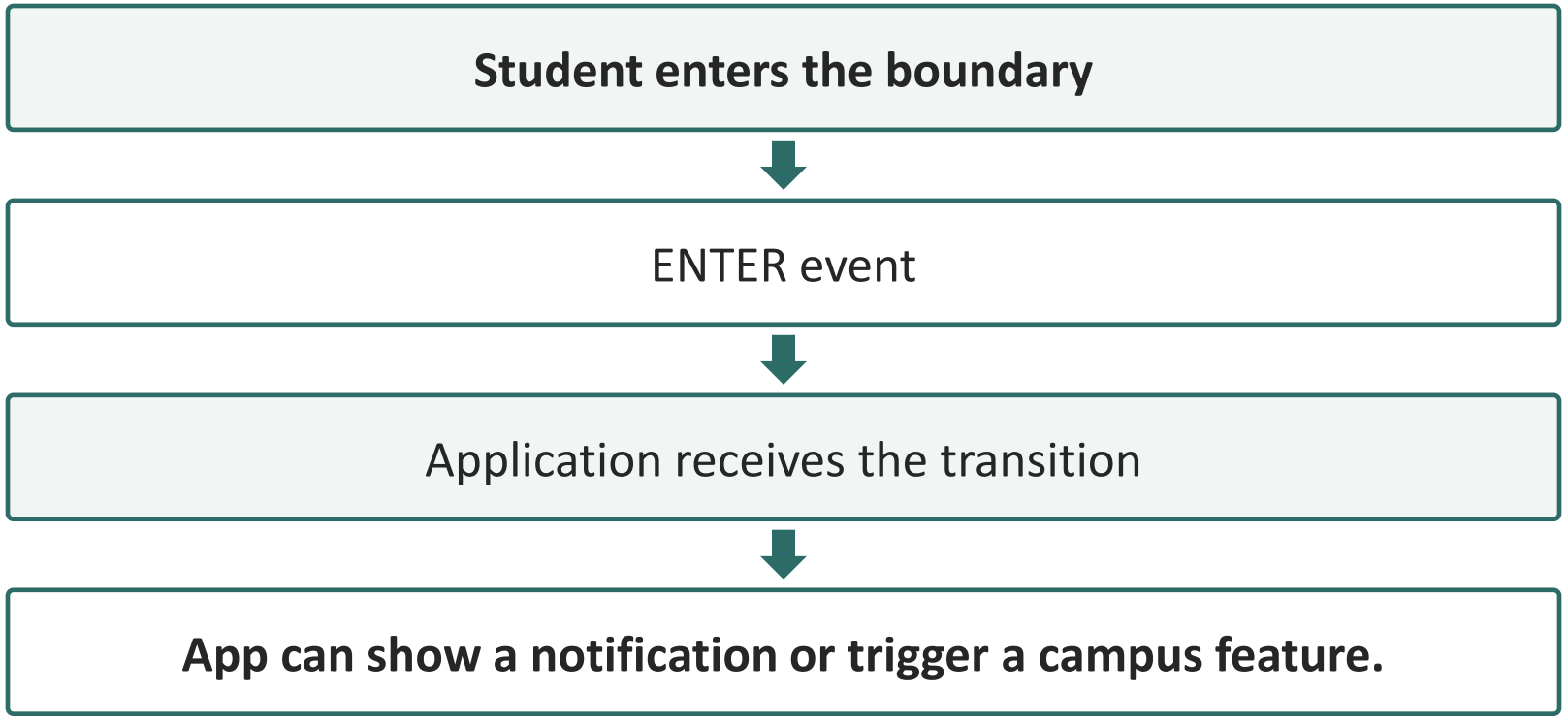
- A geofence defines a virtual geographic boundary around a real-world area.
- Instead of continuously asking where the user is, the app can react when the user enters or leaves an area.
- Typical transitions: ENTER, EXIT and DWELL.
- Examples: campus attendance, store notifications, delivery zones and smart-home automation.

How Geofencing Works



Geofencing Example: Campus

Define campus center coordinates and a radius.



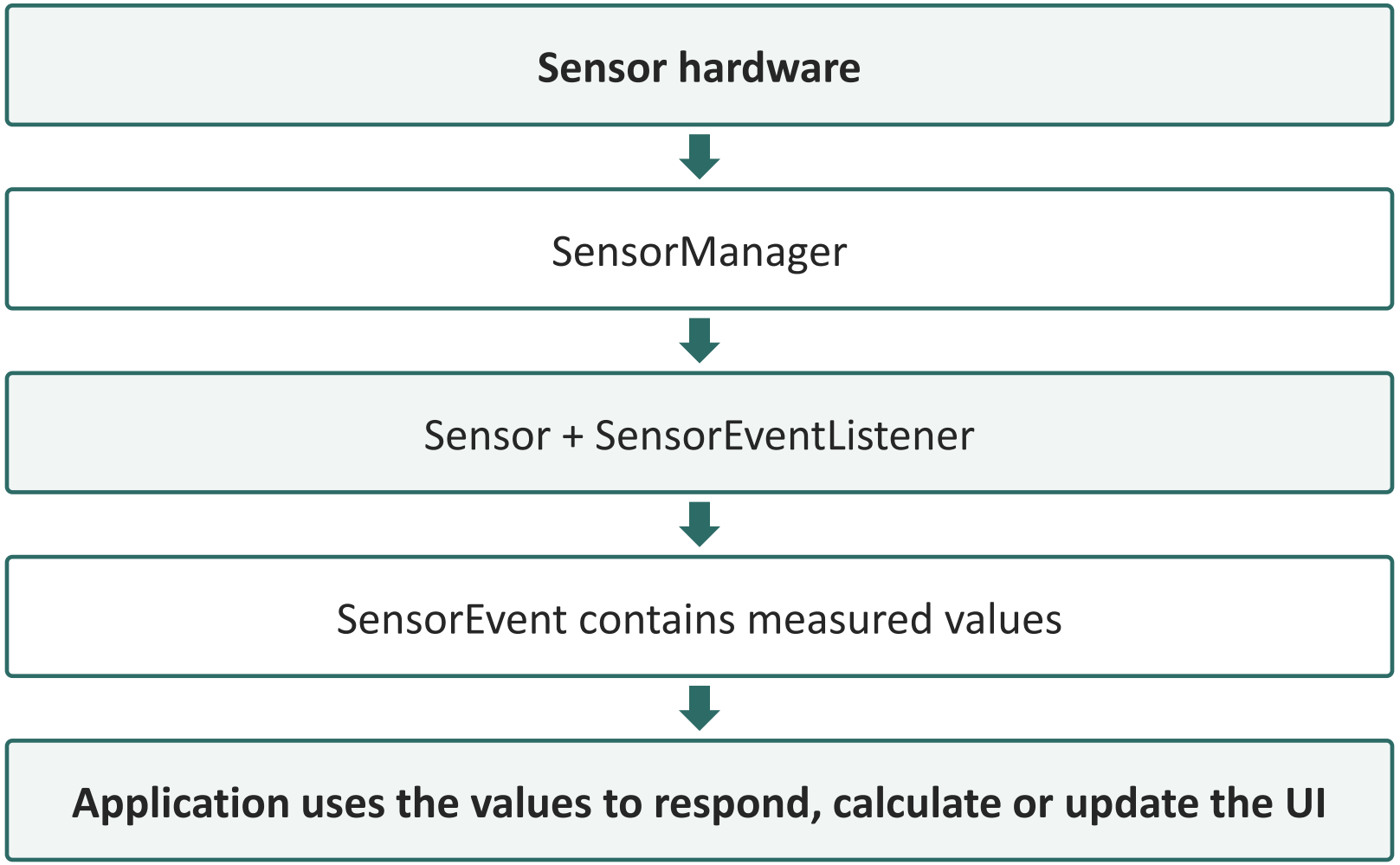
Geofencing is event-driven; it is different from continuously displaying location.

Smartphone Sensors

02

- Smartphones contain sensors that measure movement, position or environmental conditions.
- Examples include accelerometer, gyroscope, proximity, light and magnetic-field sensors.
- Android provides a common Sensor Framework for accessing supported sensors.
- Before using a sensor, check whether the device actually provides it.

Android Sensor Framework



Key Sensor APIs

SensorManager

manages access to available sensors.

Sensor

represents a specific sensor.

SensorEvent

contains sensor readings and a timestamp.

SensorEventListener

receives sensor accuracy and value changes.

Register listeners when needed and unregister them when the feature is no longer needed.

Accelerometer

- Measures acceleration along the device's X, Y and Z axes.
- Values are affected by device movement and gravity.
- Useful for movement, tilt changes and gesture-based interaction.
- Examples: shake detection, games, motion-aware interfaces and activity features.
- Think: “How is the device accelerating or moving?”

Accelerometer: X, Y, Z

X axis

left / right

Y axis

up / down

Z axis

toward / away from the device surface

Interpretation depends on Android's device coordinate system and orientation. Sensor readings are typically delivered through SensorEvent.values[].

Gyroscope

Measures angular velocity — how quickly the device is rotating. Reports rotation around the X, Y and Z axes.

Accelerometer

linear acceleration / movement

Gyroscope

rotational movement

Useful for games, AR, motion interaction and orientation-related features.

Proximity Sensor

- Detects whether an object is close to the device.
- May report distance or a near/far state depending on the sensor.
- Classic example: during a phone call, the display can respond when the device is near the user's face.
- Useful for simple context-aware interactions.

Camera + Sensor Framework

- Camera and sensors are smartphone hardware capabilities accessed through different Android APIs.
- CameraX provides a modern approach for common camera use cases.
- Camera applications need appropriate permission and lifecycle handling.
- Examples: QR scanning, document capture, image capture and augmented reality.
- Camera → visual data; Sensor Framework → sensor measurements.

Sensor Code Pattern

1. Get SensorManager.
2. Obtain the required Sensor.
3. Check whether the sensor exists.
4. Register SensorEventListener.
5. Read SensorEvent values.
6. Use the readings in the application.
7. Unregister the listener when it is no longer needed.

Design Considerations

PRIVACY

request only necessary location access and explain its purpose.

BATTERY

avoid unnecessary continuous location updates or high-frequency sensor sampling.

AVAILABILITY

not every device contains every sensor.

ACCURACY

location and sensor readings are measurements, not perfect values.

LIFECYCLE

register/unregister listeners and manage location updates appropriately.

RESILIENCE

handle denied permissions, unavailable sensors and missing location data.

Integrated Case: Smart University Campus App

Google Maps

displays campus and important locations.

Fused Location Provider

obtains the student's current position.

Geofencing

detects entry/exit from campus.

Accelerometer

detects a simple movement/shake interaction.

Together, these APIs allow the app to respond to both PLACE and DEVICE CONTEXT.

Key Takeaways

01 Android location APIs provide geographic information to applications.

02 Fused Location Provider is a practical way to obtain location in many modern apps.

03 Location permission is required and should be handled carefully.

04 Google Maps displays geographic information; it is not the same as a location API.

05 Geofencing turns geographic boundaries into application events.

06 SensorManager and SensorEventListener provide the main pattern for reading sensors.

07 Accelerometer measures acceleration; gyroscope measures rotation; proximity detects nearby objects.

08 Good mobile apps balance accuracy, privacy, battery use and device capability.

Thank You

Questions